INFORME FINAL

Agencia de turismo «Rumbo Sur»

Trabajo Práctico Obligatorio · Etapa 2: codificación, puesta a punto y testeo

Dato	Detalle
Materia	Paradigma Orientado a Objetos
Facultad	Ingeniería y Ciencias Exactas
Turno	Viernes Mañana · 2.º cuatrimestre 2026
Profesora	Maria Angela Leon
Grupo	«PATO»

Integrantes	Legajo
Callejón, Julieta Rocío	LU1208583
Farias Hiriart, Celina	LU1207439
Kincaid, Julieta	LU1216032

1. Introducción

Este informe resume el trabajo de la Etapa 2 del trabajo práctico: la codificación en Java del sistema de la agencia de turismo «Rumbo Sur» a partir del diseño de la Etapa 1. Incluye los cambios que hubo que hacer respecto del diseño original y su justificación, las dificultades encontradas, la distribución de tareas del grupo y una reflexión final.

El sistema es una aplicación de consola con menús, organizada con el patrón Modelo-Vista-Controlador y con la persistencia separada en un repositorio que guarda los datos en un archivo. Implementa los 20 casos de uso del enunciado para los cuatro roles (administrador, coordinador de operaciones, vendedor y guía).

Aspecto	Resultado
Casos de uso	20 de 20 implementados, incluidos los dos modelados en diagramas de secuencia (CU15 y CU18)
Tamaño del código	Alrededor de 65 clases, interfaces y enumeraciones; más de 8.000 líneas en el programa y unas 1.250 en las pruebas
Arquitectura	Modelo, servicios, controladores, vistas de consola y repositorio en paquetes separados
Patrones	Los tres principios SOLID (SRP, OCP, DIP) y los cuatro patrones GRASP (Experto, Creador, Controlador, Alta cohesión) del diseño, aplicados en el código
Validaciones y excepciones	Excepciones propias del negocio; los controladores las atrapan y la vista solo muestra el mensaje
Persistencia	Archivo único con serialización de objetos y guardado seguro (primero un archivo temporal)
Pruebas	98 pruebas unitarias con JUnit 5 y un plan de 41 casos de pruebas funcionales, todos con resultado conforme

2. Cambios respecto del diseño original

El diagrama de clases de la Etapa 1 fue la guía de la codificación y la mayor parte se respetó. Sin embargo, al programar las reglas de negocio aparecieron responsabilidades que el diagrama no tenía resueltas, y hubo que agregar clases y mover métodos. Cada cambio se justifica con el principio o patrón que lo motivó.

2.1 Lo que se mantuvo del diseño

Elemento del diseño de la Etapa 1	Estado en el código final
CalculadoraPrecioReserva (SRP, Alta cohesión)	Implementada tal como se diseñó: calcula el precio y las promociones fuera de Reserva.
Promocion, PromocionAnticipada y PromocionGrupo (OCP)	Implementadas con polimorfismo: cada promoción decide si es aplicable y la calculadora toma la más favorable.
CalculadorPromociones y GestorPromociones (DIP)	Implementadas: la calculadora depende de la interfaz y no de la clase concreta.
Salida.calcularOcupacion() y verificarCupo() (Experto)	Se mantienen en Salida, que conoce su cupo y sus reservas. Se sumaron cuposDisponibles() y calcularPorcentajeOcupacion().
Reserva.crearPago() (Creador)	Se mantiene. Reserva también pasó a crear sus notas de ajuste.
ControladorReservas (Controlador)	Se mantiene y se complementó con otros controladores (punto 2.2).

Elemento del diseño de la Etapa 1	Estado en el código final
Jerarquía de Personal, Tour y TourAventura, ServicioTour y ServicioContratado, Pago, Cancelacion y Devolucion	Se mantienen con la misma estructura y relaciones.

2.2 Clases y elementos nuevos

Elemento	Qué hace	Por qué se agregó
Agencia	Guarda las colecciones del sistema (tours, salidas, clientes, personal, etc.), controla las claves únicas y la numeración, y ofrece las operaciones que cruzan varias clases.	El diagrama no tenía ninguna clase que conociera todos los catálogos. Sin ella no había dónde controlar las claves únicas ni un único objeto raíz que guardar en el archivo.
ServicioAsignacion	Verifica que un guía o un vehículo estén habilitados y disponibles (categoría, capacidad, superposición de fechas).	La regla necesita conocer varias salidas a la vez, así que no corresponde a Salida ni a Guia (Experto, Alta cohesión). El diagrama tenía Guia.estaDisponible().
ServicioLiquidacion y ServicioReportes	Calculan los honorarios del guía y los reportes de ocupación, ventas, ingresos y cancelaciones.	El diagrama los ponía como métodos de Administrador. Se separaron para que esa clase no concentre cálculos (SRP).
PoliticaCancelacion	Determina qué porcentaje se devuelve según los días de anticipación y quién cancela.	Aísla una regla que puede cambiar sin tocar Reserva ni Cancelacion (SRP).
Excepciones propias (ReglaNegocioException y tres derivadas)	Representan los errores de negocio con mensajes claros.	La consigna exige validaciones y excepciones. Permiten distinguir un error de negocio de uno de programación.
ControladorAcceso, ControladorOperaciones, ControladorCatalogo y ControladorAdministracion	Un controlador por área de casos de uso, además de ControladorReservas.	El diseño solo tenía ControladorReservas. Un único controlador para los 20 casos de uso habría perdido cohesión (GRASP Controlador, Alta cohesión).
Pantalla (interfaz)	Define cómo un controlador le ordena a la vista mostrar un mensaje de éxito, un error o un aviso.	Los controladores dependen de una abstracción y no de la consola (DIP).
Vistas de consola (Consola, Listados y una vista por rol)	Leen lo que escribe el usuario y muestran resultados; no contienen reglas de negocio.	Capa Vista del patrón MVC, que el diagrama de la Etapa 1 no incluía.
RepositorioAgencia y RepositorioAgenciaArchivo	Guardan y cargan el estado completo de la agencia en un archivo.	Separan la persistencia del modelo (SRP y DIP): el modelo no sabe cómo ni dónde se guarda.
Principal y DatosIniciales	Arrancan el programa conectando las capas y cargan los datos de prueba la primera vez.	Necesarios para ejecutar el sistema y para poder probarlo sin cargar todo a mano.

2.3 Cambios en clases del diagrama

Clase	Diseño original	Código final	Motivo
Administrador y CoordinadorOperaciones	Tenían métodos (gestionarCatalogo, generarReportes, liquidarHonorarios, asignarGuia, iniciarSalida, etc.).	Quedaron como clases de identidad del rol. Las operaciones pasaron a los controladores y servicios.	Evitar que las clases del dominio coordinen casos de uso (GRASP Controlador).

Clase	Diseño original	Código final	Motivo
Salida	confirmar(), iniciar(), finalizar(), cancelar().	El estado se actualiza solo según la ocupación (se eliminó confirmar()). iniciarSalida(hoy), finalizarSalida(), cancelar(hoy, motivo) y cancelarSiNoAlcanzaMinimo(hoy) reciben la fecha. Se agregaron crearReserva(), consultarListaPasajeros() y seSuperponeCon().	Confirmación y cancelación automáticas (RN06, RN07). La fecha como parámetro permite controlar plazos y probarlos.
Reserva	registrarPago(), cancelar(), confirmar().	crearPago(), cancelar(hoy, motivo, origen), crearNotaAjuste(), admitePagos(), verificarPlazoPago(hoy), contratarServicio() y quitarServicio(). La confirmación es automática al cubrir la seña.	Reserva crea sus pagos y ajustes (Creador) y controla los plazos de pago (RN17, RF18).
Pasajero	calcularPrecio(tarifa).	El cálculo del precio quedó en CalculadoraPrecioReserva.	El precio depende de toda la reserva (tipo, tarifa, servicios y promociones): SRP y Alta cohesión.
Cancelacion y Devolucion	calcularDevolucion() y registrarDevolucion().	PoliticaCancelacion calcula el porcentaje; Cancelacion genera una devolución por cada pago y calcula el total devuelto.	Separar la regla de los datos y devolver por el mismo medio de cada pago (RN21).
Guia	estaDisponible(inicio, fin).	La disponibilidad la verifica ServicioAsignacion. Guia gestiona idiomas y categorías con protecciones (no queda sin idioma ni sin categoría).	La disponibilidad exige conocer las demás salidas.
Cliente	Tenía un atributo id.	Se identifica por su documento, que no puede repetirse.	El documento ya era único (CU14); el id no aportaba.
Tour	obtenerTarifa(fecha).	obtenerTarifaVigente(fecha), baja lógica (estaActivo, darDeBaja) y validarConfiguracion().	Un tour dado de baja no admite salidas nuevas (RF01) y solo se programan salidas de tours completos.
NotaAjuste	registrarAjuste().	Reserva la crea con crearNotaAjuste().	Creador: la reserva contiene sus ajustes.
ControladorReservas	crearReserva(), confirmarReserva() y cancelarReserva().	Es el controlador del vendedor: clientes, reservas, pasajeros, servicios, pagos, cancelaciones y ajustes.	Cubrir los casos de uso CU14 a CU19.

2.4 Cambios en las enumeraciones

Enumeración	Cambio	Motivo
TipoServicioAdicional	EXCURSION, TRASLADO_AEROPUERTO y SEGURO_VIAJE pasaron a EXCURSION, TRASLADO y SEGURO.	Nombres más cortos y generales.
OrigenCancelacion	Se agregó FALTA_PAGO a CLIENTE y AGENCIA.	El sistema anula las reservas con saldo vencido (RN17, RF18). Esa causa no la elige el vendedor y no devuelve la seña.

Las demás enumeraciones (EstadoReserva, EstadoSalida, TipoPasajero, MedioPago, TipoVehiculo, Temporada, CategoriaTour y TipoActividad) se mantuvieron como en el diagrama.

2.5 Supuestos adoptados

El enunciado indica que todo dato necesario que no figure debe completarse con supuestos. Estos son los principales:

Tema	Supuesto adoptado
Ingreso al sistema	Se ingresa con el legajo, sin contraseña. No forma parte de los 20 casos de uso y se agregó como apoyo.
Tipos de pasajero	Menor de edad: menos de 18 años. Menor de 12 años paga el 60 % y menor de 3 años no paga (enunciado). De 12 a 17 años paga la tarifa completa. Jubilado: desde los 60 años, con 10 % de descuento. La edad se calcula a la fecha de inicio de la salida.
Seña y saldo	La seña es el 30 % del total. El saldo se paga hasta 10 días antes de la salida; no se aceptan pagos fuera de ese plazo ni anteriores a la reserva, ni mayores al saldo.
Reservas	Se aceptan hasta 10 días antes de la salida, igual que el plazo del saldo. El reemplazo de pasajeros se permite hasta 3 días antes.
Ocupación y estados	La ocupación cuenta los pasajeros de las reservas pendientes, confirmadas y finalizadas; una reserva anulada libera sus lugares. La salida se confirma al llegar al cupo mínimo.
Promociones y precio	Se aplica solo la más favorable (15 % por anticipación de más de 60 días o 10 % para grupos de seis o más) sobre el total de tarifas y servicios.
Pagos con tarjeta	El recargo del 7 % por cuotas se cobra aparte: no integra el saldo, pero se devuelve si la reserva se cancela.
Cancelaciones	Si cancela el cliente se devuelve el 90 % con 30 días o más de anticipación, el 50 % con 15 a 29 días, el 20 % con 7 a 14 días y nada con menos de 7. Si cancela la agencia se devuelve todo. Si se anula por falta de pago no se devuelve la seña. La devolución se hace por el mismo medio de cada pago.
Modificar reservas	No se pueden quitar pasajeros ni servicios de una reserva que ya tiene pagos; para corregirla se registra una nota de ajuste, que solo se admite si hay pagos.
Iniciar una salida	Debe estar confirmada o completa, haber llegado su fecha de inicio, y tener guía, vehículo y la lista de pasajeros emitida. Si falta la lista, el sistema la emite.
Tours con salidas	Un tour con salidas programadas o en curso no puede darse de baja ni cambiar de duración, de cupos ni de condiciones.
Honorarios del guía	Monto fijo por día multiplicado por los días de la salida, más el 5 % de los servicios adicionales (a precio de lista) de las reservas finalizadas.
Reportes	Una venta es una reserva confirmada o finalizada cuya fecha cae en el período. Los ingresos netos son lo cobrado menos lo devuelto.
Claves únicas	Reservas y salidas por número correlativo, clientes por documento, personal por legajo, tours por código, vehículos por patente y proveedores por CUIT.

2.6 Cómo se reflejan SOLID y GRASP en el código final

Algunos ejemplos:

Principio o patrón	Dónde está	Evidencia
SRP	CalculadoraPrecioReserva, PoliticaCancelacion, ServicioAsignacion, ServicioLiquidacion, ServicioReportes y RepositorioAgenciaArchivo	Cada clase tiene una sola razón de cambio: el precio, la devolución, las asignaciones, la liquidación, los reportes o el guardado.

Principio o patrón	Dónde está	Evidencia
OCP	Promocion, PromocionAnticipada y PromocionGrupo	Se puede agregar una promoción nueva sin modificar la calculadora.
DIP	CalculadorPromociones, Pantalla y RepositorioAgencia	La calculadora, los controladores y el arranque dependen de interfaces; GestorPromociones, Consola y RepositorioAgenciaArchivo son las implementaciones.
Experto	Salida.calcularOcupacion() y verificarCupo(); Reserva.calcularSaldo(); Pasajero.calcularEdad()	Cada cálculo está en la clase que tiene los datos necesarios.
Creador	Salida.crearReserva(); Reserva.crearPago() y crearNotaAjuste(); Cancelacion y sus devoluciones	El objeto contenedor crea los objetos que contiene.
Controlador	ControladorAcceso, ControladorOperaciones, ControladorReservas, ControladorCatalogo y ControladorAdministracion	Reciben las solicitudes de la vista, controlan el rol, delegan en el modelo y le ordenan a la pantalla mostrar el resultado.
Alta cohesión	Servicios de cálculo separados de Reserva y de Administrador	Reserva administra sus datos, pasajeros, pagos y estado; los cálculos económicos están en clases propias.

3. Dificultades encontradas

Estas fueron las principales dificultades técnicas y cómo se resolvieron:

Dificultad	Cómo se resolvió
Dónde atrapar los errores en el patrón MVC. En las primeras versiones cada vista atrapaba las excepciones, y la presentación se mezclaba con el control de las reglas.	Los controladores atrapan las excepciones de negocio y le ordenan a la interfaz Pantalla mostrar el mensaje. Las vistas no tienen try/catch: solo leen datos y muestran resultados. Los controladores devuelven verdadero o falso, o los datos (o null si falló).
Reglas que dependen de la fecha. Los plazos de pago, las reservas, el cupo mínimo, las cancelaciones y el inicio de las salidas dependen del día. Si el modelo consultara el reloj, no se podrían probar.	La fecha de hoy se pasa como parámetro a los métodos que la necesitan. Las pruebas usan la fecha que quieren y los datos de prueba se calculan con fechas relativas al día de la primera ejecución.
Coherencia entre Salida y Reserva. La ocupación, la confirmación, el cupo completo, la liberación de lugares al anular y la cancelación por cupo mínimo dependen unas de otras.	El estado de la salida se recalcula en un único lugar a partir de la ocupación. La Agencia ofrece verificarPlazosPago() y verificarCuposMinimos(), que se ejecutan al abrir el programa y antes de iniciar una salida.
Persistencia con serialización. Al modificar una clase del modelo, el archivo guardado dejaba de poder leerse, y había riesgo de pisar los datos.	Las clases del modelo son Serializable. Si el archivo no puede leerse, el programa lo avisa y se cierra sin sobrescribirlo. El guardado escribe primero un archivo temporal y después lo reemplaza. Se documentó cómo reiniciar los datos.
Modificaciones a medias. En cambios con varios datos (por ejemplo nombre y descripción de un tour), si el segundo dato era inválido el primero ya había cambiado.	Se valida todo primero con un objeto descartable y recién después se modifica el objeto real.
Reglas que cruzan varias clases. La disponibilidad de guías y vehículos, la capacidad frente al cupo y las categorías del guía frente a sus salidas no pertenecen a una sola clase.	Se creó la clase Agencia, que conoce todos los catálogos, y ServicioAsignacion, que concentra las verificaciones de asignación.

Dificultad	Cómo se resolvió
Precios y promociones. Combinar el tipo de pasajero, la temporada, los servicios, la promoción más favorable y el recargo de tarjeta, que no integra el saldo pero se devuelve.	CalculadoraPrecioReserva y GestorPromociones concentran el cálculo, y el total se recalcula después de cada cambio de la reserva. Las pruebas verifican montos concretos.
Ambigüedades del enunciado. Por ejemplo, desde qué edad se es jubilado, qué porcentaje se devuelve en cada tramo o qué se considera una venta.	Se tomaron supuestos explícitos y se documentaron (punto 2.5), para que el comportamiento sea el mismo en el código, las pruebas y la documentación.
Datos de prueba coherentes. Armar un historial válido (reservas anteriores a la salida, pagos dentro del plazo, una salida finalizada y una reserva anulada) sin romper las propias reglas.	DatosIniciales usa el propio modelo para crear todo, de modo que las reglas se cumplen también en los datos de prueba, con fechas calculadas desde el día de la carga.
Armar las pruebas. Cada prueba necesitaba muchas clases relacionadas (tour, tarifas, salida, cliente, vendedor, guía, vehículo).	Se creó un escenario reutilizable que prepara la agencia, y una pantalla falsa para probar los controladores. El plan funcional se ejecutó siempre partiendo de datos recién cargados, para que los casos fueran independientes.
Una consola cómoda de usar. Con 20 casos de uso, los menús quedaban largos y era fácil equivocarse al escribir códigos.	Los menús se organizaron en submenús por área. Siempre que se puede se elige de una lista numerada y se ofrecen solo las opciones válidas (por ejemplo, para cobrar, solo las reservas con saldo).

4. Distribución de tareas

Área de trabajo	Responsable principal	Colaboración
Modelo de reservas: Reserva, Pasajero, Pago, Cancelacion y NotaAjuste; promociones y CalculadoraPrecioReserva	Callejón, Julieta Rocío	Kincaid, Julieta
Controlador y vista del vendedor (clientes, reservas, pagos, cancelaciones y ajustes)	Callejón, Julieta Rocío	Kincaid, Julieta
Modelo de catálogo y operaciones: Tour, Tarifa, Actividad, Salida, Guia, Vehiculo y Proveedor	Farias Hiriart, Celina	Callejón, Julieta Rocío
Agencia y servicios de asignación, liquidación y reportes	Farias Hiriart, Celina	Callejón, Julieta Rocío
Controladores y vistas del administrador y del coordinador	Farias Hiriart, Celina	Kincaid, Julieta
Persistencia en archivo (repositorio y serialización) y carga de datos de prueba	Kincaid, Julieta	Farias Hiriart, Celina
Consola, listados, ingreso al sistema y vista del guía	Kincaid, Julieta	Callejón, Julieta Rocío
Pruebas unitarias del modelo y de los servicios	Callejón, Julieta Rocío	Farias Hiriart, Celina
Plan de pruebas funcionales y README	Kincaid, Julieta	Farias Hiriart, Celina
Diagrama de clases final, documentación e informe	Las tres integrantes	—
Repositorio de GitHub: commits, ramas y tags de cada etapa	Las tres integrantes	—

5. Reflexión del grupo

Este trabajo nos permitió ver de punta a punta cómo se pasa de un enunciado a un sistema que funciona. Lo que más nos enseñó fue que aplicar los principios de diseño no es algo que se hace una vez: cada regla nueva nos obligó a preguntarnos de quién era la responsabilidad. El diagrama de la Etapa 1 resultó una buena guía, pero al programar descubrimos huecos, como la necesidad de una clase que conociera todos los catálogos o de servicios para las reglas que involucran varias salidas. Adaptar el diseño en lugar de forzarlo fue parte del aprendizaje.

La separación en capas fue lo que más valor tuvo. Con el modelo sin ninguna dependencia de la consola, los controladores atrapando las excepciones y las vistas limitadas a leer y mostrar, pudimos cambiar los menús varias veces sin tocar las reglas de negocio, y probar esas reglas sin pasar por la pantalla. Lo mismo ocurrió con pasar la fecha como parámetro: complicó un poco las firmas de los métodos, pero hizo posible probar los plazos sin esperar días.

También aprendimos a tomar decisiones donde el enunciado no definía nada, y a dejarlas escritas. Anotar los supuestos nos evitó discutir de nuevo lo mismo y hace que el comportamiento sea igual en el código, en las pruebas y en la documentación.

Reconocemos estas limitaciones del trabajo, que serían los siguientes pasos:

- La interfaz es solo de consola. La interfaz gráfica con Swing era opcional y no la desarrollamos.
- El ingreso se hace solo con el legajo, sin contraseña, y habría que agregar autenticación.
- La persistencia por serialización es simple, pero sensible a los cambios en las clases. Una base de datos sería más robusta para un uso real.
- No hay búsquedas por otros criterios (por ejemplo, reservas de un cliente) ni modificación de la comisión o el honorario desde los menús.
- Algunos detalles de presentación se pueden pulir, como el formato de una fecha en un mensaje de error.

En conjunto, el trabajo nos dejó una idea concreta de lo que significa diseñar con responsabilidades claras, y una base de código que podemos seguir ampliando.

Anexo A. Correspondencia entre casos de uso y código

Para cada caso de uso, el controlador que lo coordina y las clases que implementan la regla.

CU	Controlador y operaciones	Modelo o servicio	Menú
CU01	ControladorCatalogo: registrarTour, registrarTourAventura, modificarDatosTour, modificarDuracionTour, modificarCuposTour, darDeBajaTour	Tour, TourAventura, Agencia	Administrador, Tours
CU02	ControladorCatalogo: agregarActividad, modificarHorarioActividad, modificarOrdenActividad, quitarActividad	Tour, Actividad	Administrador, Itinerario
CU03	ControladorCatalogo: registrarProveedor, modificarProveedor, asociarProveedor	Proveedor, Actividad	Administrador, Proveedores
CU04	ControladorCatalogo: agregarTarifa, modificarPrecioTarifa, modificarFechasTarifa	Tour, Tarifa	Administrador, Tarifas
CU05	ControladorCatalogo: registrarServicioAdicional, ofrecerServicioEnTour, cambiarObligatoriedadServicio	ServicioAdicional, ServicioTour	Administrador, Servicios
CU06	ControladorCatalogo: registrarVehiculo, modificarVehiculo	Vehiculo, Agencia	Administrador, Vehículos

CU	Controlador y operaciones	Modelo o servicio	Menú
CU07	ControladorCatalogo: registrarVendedor, registrarGuia, modificarDatosPersonal, gestión de idiomas y categorías	Personal y sus subclases, Guia	Administrador, Personal
CU08	ControladorAdministracion: liquidarHonorarios	ServicioLiquidacion, LiquidacionGuia	Administrador, Liquidar honorarios
CU09	ControladorAdministracion: reporteOcupacion, contarVentas, calcularIngresosNetos, reporteCancelaciones	ServicioReportes	Administrador, Reportes
CU10	ControladorOperaciones: programarSalida	Salida, Agencia	Coordinador, Programar una salida
CU11	ControladorOperaciones: asignarGuia, asignarVehiculo	ServicioAsignacion, Salida	Coordinador, Asignar guía y vehículo
CU12	ControladorOperaciones: emitirListaPasajeros, iniciarSalida	Salida (iniciarSalida)	Coordinador, Iniciar una salida
CU13	ControladorOperaciones: finalizarSalida	Salida (finalizarSalida), Reserva	Coordinador, Finalizar una salida
CU14	ControladorReservas: registrarCliente	Cliente, Agencia	Vendedor, Clientes
CU15	ControladorReservas: crearReserva, agregarPasajero	Salida.crearReserva, Reserva, CalculadoraPrecioReserva	Vendedor, Reservas
CU16	ControladorReservas: registrarPago	Reserva.crearPago, Pago	Vendedor, Pagos
CU17	ControladorReservas: agregarPasajero, eliminarPasajero, reemplazarPasajero, contratarServicio, quitarServicio	Reserva, ServicioContratado	Vendedor, Reservas
CU18	ControladorReservas: cancelarReserva	Reserva.cancelar, PoliticaCancelacion, Cancelacion, Devolucion	Vendedor, Cancelar una reserva
CU19	ControladorReservas: registrarNotaAjuste	Reserva.crearNotaAjuste, NotaAjuste	Vendedor, Nota de ajuste
CU20	ControladorOperaciones: consultarListaPasajeros	Salida.consultarListaPasajeros	Guía, Lista de pasajeros